Project 2 Assessment Guide

2610EST3000 · Project 2: Recursive Self-Improvement Strategies (100 pts)

This cover page states how the report that follows is graded. It is an instructor annotation: it is not part of the report, does not count toward the 9-page body budget, and the report begins on the next page with its usual layout and page numbering.

About the project

Background. A recursive self-improvement (RSI) engine runs research cycles autonomously: each cycle proposes a single-variable experiment, runs it under a fixed budget, records the outcome, and may rewrite the engine's own strategy through a guard that admits only edits with immediate compute savings.

Problem. Naive strategies waste most cycles on re-exploration and no-change experiments, so accumulated knowledge does not raise the research yield and the engine plateaus early.

Requirements. Without editing the frozen engine, design, motivate, implement and evaluate strategies for proposal generation, result diagnosis and guarded self-editing; report evidence against matched baselines over fixed seeds, with the fate of every proposed self-edit accounted for.

Learning outcomes. Formulate a research bottleneck as a learning problem; connect the design to course concepts; run controlled experiments with honest statistics; and communicate results with evidence-calibrated claims.

Component	Weight	What earns the marks
Literature survey	10%	**Coverage:** the foundations of recursive self-improvement and automated research, plus the most relevant recent work. **Precision:** each citation supports the exact claim it is attached to; no padding. **Synthesis:** approaches are grouped, compared and contrasted, not listed. **Gap:** the limitation this submission targets is named, with evidence from the literature. **Positioning:** what is reused from the given engine is separated from what is new here.
Reasoning & insight	30%	**Diagnosis:** the bottleneck (e.g., wasted cycles, no-op edits) is identified from measured behaviour, not intuition. **Motivation:** every strategy states the hypothesis it tests and why its mechanism should close the gap. **Course linkage:** explicit use of ML concepts — exploration–exploitation, bias–variance, credit assignment, MDP/bandit formulation, generalization, statistical testing. **Formalization:** the decision problem (what to propose, keep, or edit) is posed with assumptions stated. **Theory:** derivations, propositions, or a compute/budget argument from which the design follows. **Alternatives:** rejected designs and reasons; predictions that the experiments can falsify.
Experiments	30%	**Implementation:** every claimed strategy exists in runnable code, integrated within the frozen engine and trust boundary. **Protocol:** naive and matched baselines, equal cycle budgets, multiple seeds fixed in advance, metric defined before the runs. **Main results:** seed mean $\pm$ standard deviation against baselines; effect sizes quantified. **Reproducibility:** exact commands, configs, seed lists and the self-edit ledger; a clean checkout reproduces the headline number. **Honesty:** negative and failed runs reported; compute cost accounted; no cherry-picking.
Result analysis	30%	**Attribution:** ablations isolate which component carries the gain. **Mechanism:** gains and failures explained causally — overfitting to noise, distribution shift, non-stationarity, exploration collapse. **Dynamics:** keep-rate versus budget, per-task and per-cycle behaviour, where gains and stagnation arise. **Failure anatomy:** rejected proposals, discarded self-edits and guard arbitrations are taxonomized. **Calibration:** limitations and threats to validity stated; conclusions match the strength of the evidence.

Marking. Component marks are weighted by the percentages above and summed to 100; within a component, credit follows the strength of verifiable evidence, from fully met (85–100%) through met but thin (70–84%) and partial or asserted (60–69%) to unsupported (<60%).

Deliverables (Canvas). The report PDF (this template; at most 9 pages of body, excluding references, the checklist and the appendix) and a runnable code archive implementing every reported strategy, whose clean checkout reproduces the headline number, together with seed lists, run logs and the self-edit ledger.

Policies. Individual work, one author per submission; the frozen engine and trust boundary may not be modified; every claim must trace to actual runs and real, verifiable citations — fabricated results or references score zero; LLM use in methods and in writing disclosed per the checklist; deadline and late policy per the Canvas announcement.

[TODO: Title: state the strategy you built and its measurable payoff in one line; avoid "A Study of" titles]

Firstname Lastname
Student ID: [TODO: 2024XXXXXX]
Westlake University
firstname.lastname@westlake.edu.cn

Abstract

Recursive self-improvement (RSI) engines can run autonomously, but with naive strategies their research yield plateaus quickly. **[TODO: 1-2 sentences: name the specific bottleneck you attack, e.g. "proposals ignore accumulated evidence, so most cycles are wasted on re-exploration".]** Our key insight is that **[TODO: one-sentence core insight]**. We implement **[TODO: the strategy bundle, corresponding to whichever of C-1/C-2/E-2/E-3 you completed]** on top of a frozen three-loop RSI engine, and evaluate on **[TODO: task name(s)]** under matched cycle budgets. **[TODO: Method name]** improves the final best result from **[TODO: X]** to **[TODO: Y]** ($\pm$ **[TODO: seed std]** over **[TODO: N]** seeds), raises proposal keep-rate from **[TODO: A%]** to **[TODO: B%]**, and produces **[TODO: Z]** promoted self-edits that survive guard arbitration. Ablations show that **[TODO: which component carries the gain]**.

1 Introduction

[TODO: Open with the RSI engine running a research task: the single-variable experiment loop, the accumulating knowledge base, self-edits that raise research yield. Two or three sentences of minimal necessary background; assume an ML-literate reader who has never heard of this system.]

[TODO: Be concrete about the naive failures: the fixed grid walk ignores evidence (cite your own measurements on the public tasks: what fraction of cycles were no-change waste; after which cycle the best result plateaus; where it falls short of the Claude Code baseline in `baselines/public_tasks_baseline.md`).]

[TODO: One sentence: "with the engine loop and trust boundary, and the five public task scorers, frozen, make the strategy layer open a reproducible yield gap over the naive baseline and close the gap to the Claude Code baseline."]

[TODO: At least two: (i) causal attribution of keep/discard decisions under observation noise; (ii) self-edits must pass the guard without breaking determinism; add the ones you actually hit.]

[TODO: One sentence per challenge mapping to a strategy module, with a forward reference to §4.]

Our contributions are:

- **[TODO: Contribution 1: a strategy plus its one-line mechanism (maps to C-1/E-3/...).]**
- **[TODO: Contribution 2: same pattern.]**

- **[TODO: Contribution 3: an empirical finding with numbers, e.g. "evidence-guided proposal doubles keep-rate at equal budget (Table 1)".]**
- **[TODO: (Optional) a negative result or analysis contribution; negative results are respected at AI venues.]**

2 Related Work

Self-referential and self-improving systems. **[TODO: From the theoretical origin [Schmidhuber, 2009] to LLM-era self-rewriting [Zelikman et al., 2024, Zhang et al., 2025]. Point out: those works rewrite *task-solving code*; we rewrite the *research strategy itself*, under a guard.]**

LLM agents for research automation. **[TODO: [Lu et al., 2024, Huang et al., 2024, Wijk et al., 2024]: the state of end-to-end research agents and their evaluation. Point out: they use the LLM as the engine; we freeze the engine and make the research strategy itself the replaceable, measurable object.]**

Automated algorithm discovery. **[TODO: [Real et al., 2020, Yang et al., 2024, Novikov et al., 2025]: searched/evolved algorithms often beat hand design but demand large compute and human-in-the-loop evaluation. Point out: we study the same question under CPU-second, deterministic, reproducible constraints.]**

Positioning. **[TODO: Two or three closing sentences: relative to everything above, what is the unique coordinate of this work (frozen engine + strategy-layer science + strict keep/discard causal measurement)?]**

3 The Base System

[TODO: Two or three sentences: given a task bundle (one editable file + one gate metric + a fixed budget), the engine runs three nested loops; reference Figure 1.]

Loops. **[TODO: Fast: propose → implement → run → finalize → analyze, single-variable experiments. Middle: the benchmark is frozen within an epoch and upgraded across epochs; self-edits are arbitrated here. Slow: hypotheses succeed each other on a typed-edge question graph; saturated questions spawn children.]**

Frozen contracts and the trust boundary. **[TODO: The concept contracts, the verdict trichotomy, the guard layers; emphasize that self-edits may only land in the sentinel region of `state/strategy.py`, under the applying → observing → kept|reverted lifecycle with immediate re-entry.]**

Strategy seams. **[TODO: A small table: seam name / Protocol / naive default behavior / where your implementation lives (section pointer). This table is the bridge from Section 3 to Section 4 and reviewers respond well to it.]**

4 Method: Strategies Behind the Seams

[TODO: Open with two or three overview sentences: which strategies you implemented and how they cooperate (e.g. the proposer generates candidates, diagnostics decides when to upgrade the epoch, the improver turns waste signals into strategy edits). Then one subsection per strategy.]

4.1 [TODO: Strategy name, e.g. Evidence-Guided Proposal Ranking] (C-1)

Motivation. **[TODO: The concrete naive failure: fixed grid, ignores KB evidence → cite your measurement (e.g. what fraction of the naive run's 292 cycles were repeated or wasted proposals).]**

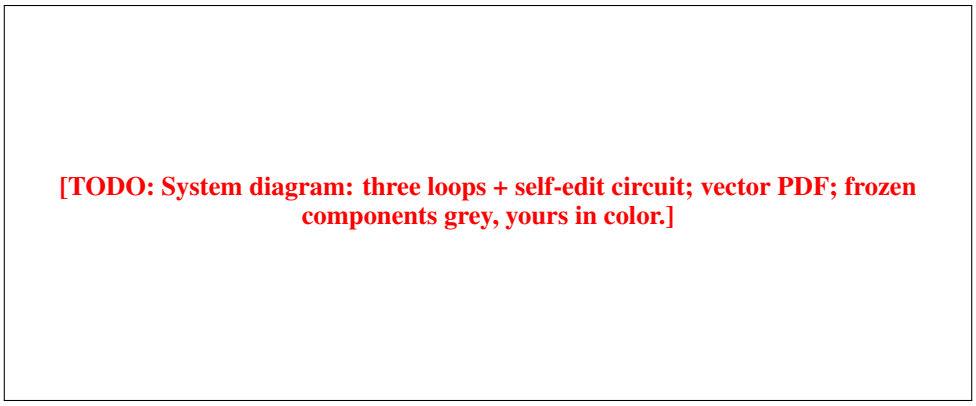


Figure 1: The three loops and the self-edit circuit. **[TODO: Make the caption complete: one clause per loop, and mark which components are frozen (grey) versus implemented in this work (color).]**

Mechanism. **[TODO: Precisely: how candidates are ranked from `evidence_for` / `best_result` / `uncertainty`; how the value grid adapts. Provide an equation or pseudocode:]**

Constraints honored. **[TODO: Determinism (the seeded RNG as the only randomness source), unchanged Protocol signatures, no touching frozen components – one sentence each.]**

4.2 [TODO: Strategy name] (C-2 / E-3 / ...)

[TODO: Follow the same three-part pattern.]

5 Experiments

5.1 Setup

Tasks and metrics. **[TODO: The five public tasks (`battle-of-sexes`, `prisoners-dilemma`, `colonel-blotto`, `house-price`, `breakout-lite`): one-line task definition + the gate metric (higher/lower is better) + where the headroom comes from. Development tasks `quad-optim`/`iris-ood` may be mentioned but do not replace the five.]**

Baselines. **[TODO: The given naive strategy, the noop-improver control, the Claude Code strong baseline (`baselines/public_tasks_baseline.md`), and one ablation slot per strategy of yours. Explain why these are fair opponents.]**

Protocol. **[TODO: Cycles, seeds (N=[TODO:]), the determinism guarantee, hardware (CPU model suffices), wall-clock per run. Command-level reproducibility goes to Appendix A.]**

5.2 Main Results

[TODO: Lead with the conclusion sentence for Table 1 ("X outperforms naive by ..."), then interpret column by column; never just drop a table. Every claim must cite a specific table/figure number.]

6 Results

6.1 Performance

[TODO: Research question: what is the overall performance across tasks? Show the main metric (e.g., task scores), with error bars. One-sentence answer: the strategy improves yield by X% over baseline.]

Table 1: Main results: final best result and keep-rate, mean ± std over N seeds. **[TODO: State budget and seed count in the caption; the table must be self-contained. Bold the best entry only when its gap over the runner-up is significant.]**

Method	Final best ↑	Keep-rate ↑
Naive (given)	**[TODO:]**	**[TODO:]**
Noop-improver ctrl.	**[TODO:]**	**[TODO:]**
Ours (self-edit ON)	**[TODO:]**	**[TODO:]**
Ours (self-edit OFF)	**[TODO:]**	**[TODO:]**
Claude Code baseline	**[TODO:]**	—

6.2 Anatomy of a Promoted Self-Edit

[TODO: The assignment's core requirement, told as a timeline: which waste pattern appeared at cycle N → which edit the improver proposed (quote the key before/after fragment of `state/strategy.py`) → each guard layer's verdict → the M-cycle observing-window comparison → the kept/reverted ruling. Pair it with a trajectory figure that marks the promotion moment with a vertical line.]

6.3 Failure Modes

[TODO: Document at least one real failure: improver proposals that get reverted, noise-induced false keeps, question-graph evolution walking into a dead end. If you omit negative results, reviewers will ask.]

6.4 Cost-Effectiveness

[TODO: Extra wall-clock per cycle, token spend (if the LLM seam is used), and whether determinism is preserved.]

6.5 Module Ablations

[TODO: Research question: which component produces the main gain? Table: full system versus one component removed at a time. When interpreting, distinguish "significant drop" from "within noise".]

7 Discussion

Limitations. **[TODO: Two or three genuine limitations, each with one sentence on the boundary and one on a path forward.]**

Broader impact. **[TODO: The risks of self-improving engines (uncontrolled self-modification, metric gaming) and this work's corresponding designs (trust boundary, default-reject guard, meltdown brake, deterministic auditing).]**

8 Conclusion

[TODO: Three or four closing sentences.]

References

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning (ICML)*, 2024.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Alexander Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Esteban Real, Chen Liang, David So, and Quoc Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *International Conference on Machine Learning (ICML)*, 2020.

Jürgen Schmidhuber. Ultimate cognition à la gödel. In *Cognitive Computation*, 2009. The Gödel machine: a self-referential universal problem solver making provably optimal self-improvements.

Hjalmar Wijk et al. RE-Bench: Evaluating frontier AI r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Self-taught optimizer (STOP): Recursively self-improving code generation. In *International Conference on Learning Representations (ICLR)*, 2024.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO: [Yes/No/NA]]**

   Justification: **[TODO: Point to the contribution list in §1 and the corresponding evidence in §5.]**

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO: [Yes]]**

   Justification: **[TODO: Point to §7.]**

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO: [NA] – this is empirical work; if you make any theoretical claim, point to its proof here.]**

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

   Answer: **[TODO: [Yes]]**

   Justification: **[TODO: Point to Appendix A: commands, seeds, configuration. The system is deterministic – same seed, same result.]**

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code?

   Answer: **[TODO: [Yes/No]]**

   Justification: **[TODO: State where the code and data live (course repository or public link). This is a course submission – no anonymization is needed.]**

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details?

Answer: **[TODO: [Yes]]**

Justification: **[TODO: Point to §5.1 and the appendix hyperparameter table.]**

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about statistical significance?

   Answer: **[TODO: [Yes]]**

   Justification: **[TODO: Mean ± std over N seeds; name the significance test used.]**

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources needed?

   Answer: **[TODO: [Yes]]**

   Justification: **[TODO: CPU model, wall-clock per run, total compute hours.]**

9. **Code of ethics**

   Question: Does the research conform to the NeurIPS Code of Ethics?

   Answer: **[TODO: [Yes]]**

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts?

    Answer: **[TODO: [Yes]]**

    Justification: **[TODO: Point to the broader-impact paragraph in §7.]**

11. **Safeguards**

    Question: Does the paper describe safeguards for responsible release of high-risk models or data?

    Answer: **[TODO: [Yes/NA]]**

    Justification: **[TODO: The guard, trust boundary, and meltdown brake are the safeguards; point to §3 and §7.]**

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (code, data, models) properly credited, and are licenses and terms of use respected?

    Answer: **[TODO: [Yes]]**

13. **New assets**

    Question: Are new assets introduced in this paper well documented, and is the documentation alongside the assets?

    Answer: **[TODO: [Yes/NA]]**

14. **Crowdsourcing and research with human subjects**

    Answer: **[TODO: [NA]]**

15. **Institutional review board (IRB) approvals**

    Answer: **[TODO: [NA]]**

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods?

    Answer: **[TODO: [Yes/NA]]**

    Justification: **[TODO: If your strategies use the `rsi.llm` seam, state the model, the role of each prompt, and how caching preserves determinism; also disclose any LLM assistance in writing (NeurIPS requires disclosure, and LLMs cannot be authors).]**

A Reproducibility Details

Environment. **[TODO: OS, Python version, `environment.yml`; CPU model and memory.]**

Exact commands. **[TODO: The full command behind every row of the main table, for example:]**

Seeds. **[TODO: The full seed list; state that seeds were fixed in advance, not selected post hoc.]**

Configuration. **[TODO: The full config.yaml or its diff from default; every non-default hyperparameter.]**

Self-edit ledger. **[TODO: The complete ledger of promoted and reverted edits (terminal states), backing the causal story in §6.2.]**

B Extended Results

[TODO: What did not fit in the body: per-seed trajectories, additional ablations, question-graph end states, the full failure-case catalog. Reviewers are not obliged to read the appendix – every core conclusion must stand in the body alone.]